

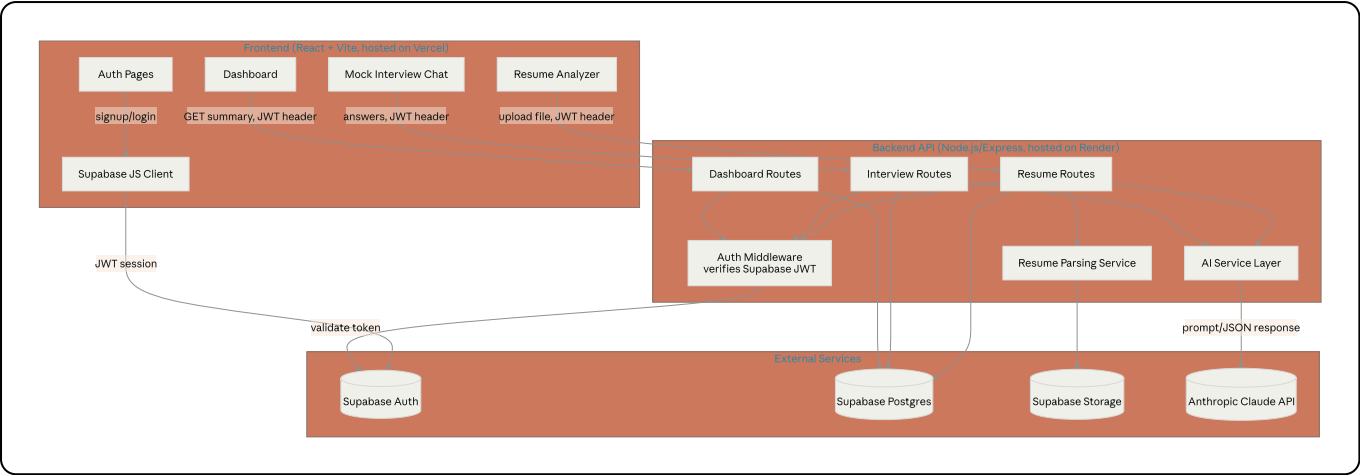
ARCHITECTURE.md — PrepGenie

1. Final Tech Stack & Justification

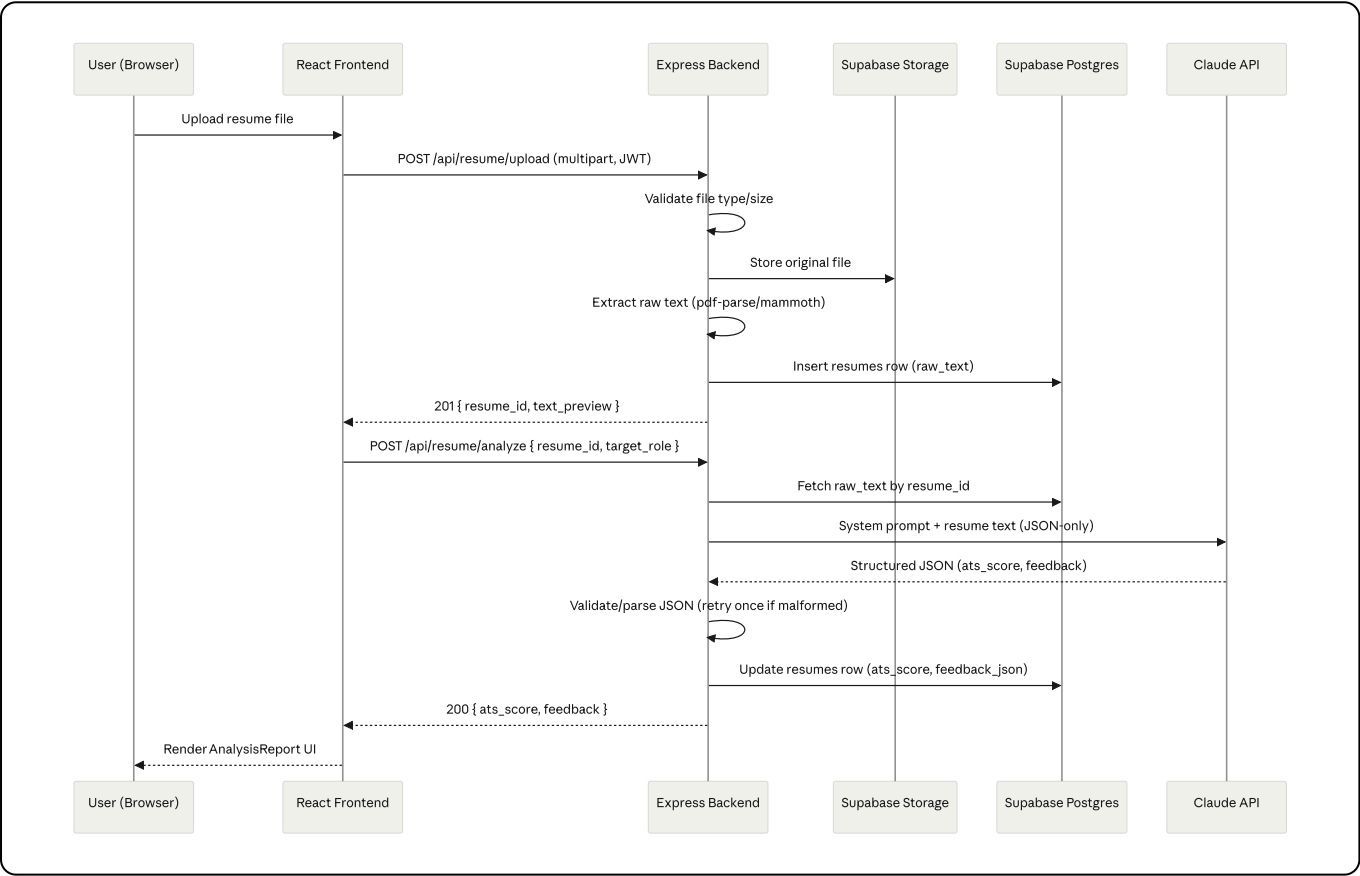
Layer	Technology	Justification
Frontend	React 18 (Vite)	Fast dev server, industry-standard, huge interview relevance
Styling	Tailwind CSS + shadcn/ui	Rapid, consistent, accessible UI without custom CSS overhead
Charts	Recharts	Declarative React charting, sufficient for line/bar/radar needs
Animations	Framer Motion	Lightweight, production-grade micro-interactions
Routing	React Router v6	Standard SPA routing, protected route support
Backend	Node.js + Express	JS-consistency across stack, minimal boilerplate, huge ecosystem
Database	PostgreSQL (via Supabase)	Relational integrity needed (users → resumes → sessions), free tier generous
Auth	Supabase Auth	Free, handles email + Google OAuth, issues JWTs consumed by backend
File Storage	Supabase Storage	Free bucket storage for uploaded resumes, integrates with Auth policies
AI Provider	Anthropic Claude API (claude-sonnet-5)	Strong structured-output (JSON) reliability, good reasoning for scoring rubrics
File Parsing	pdf-parse, mammoth	Lightweight, no external service dependency, free
Hosting (FE)	Vercel (free tier)	Zero-config React/Vite deploys, auto HTTPS, preview URLs
Hosting (BE)	Render (free tier)	Simple Node deploys, env var management, auto-restart
Notifications	react-hot-toast	Lightweight, no backend dependency

Why this stack overall: Every service used has a genuinely free tier suitable for a portfolio project, the stack is 100% JavaScript (lower context-switching cost for a B.Tech IT student), and each piece maps directly to a well-known interview talking point (auth, file handling, LLM integration, relational DB design, deployment).

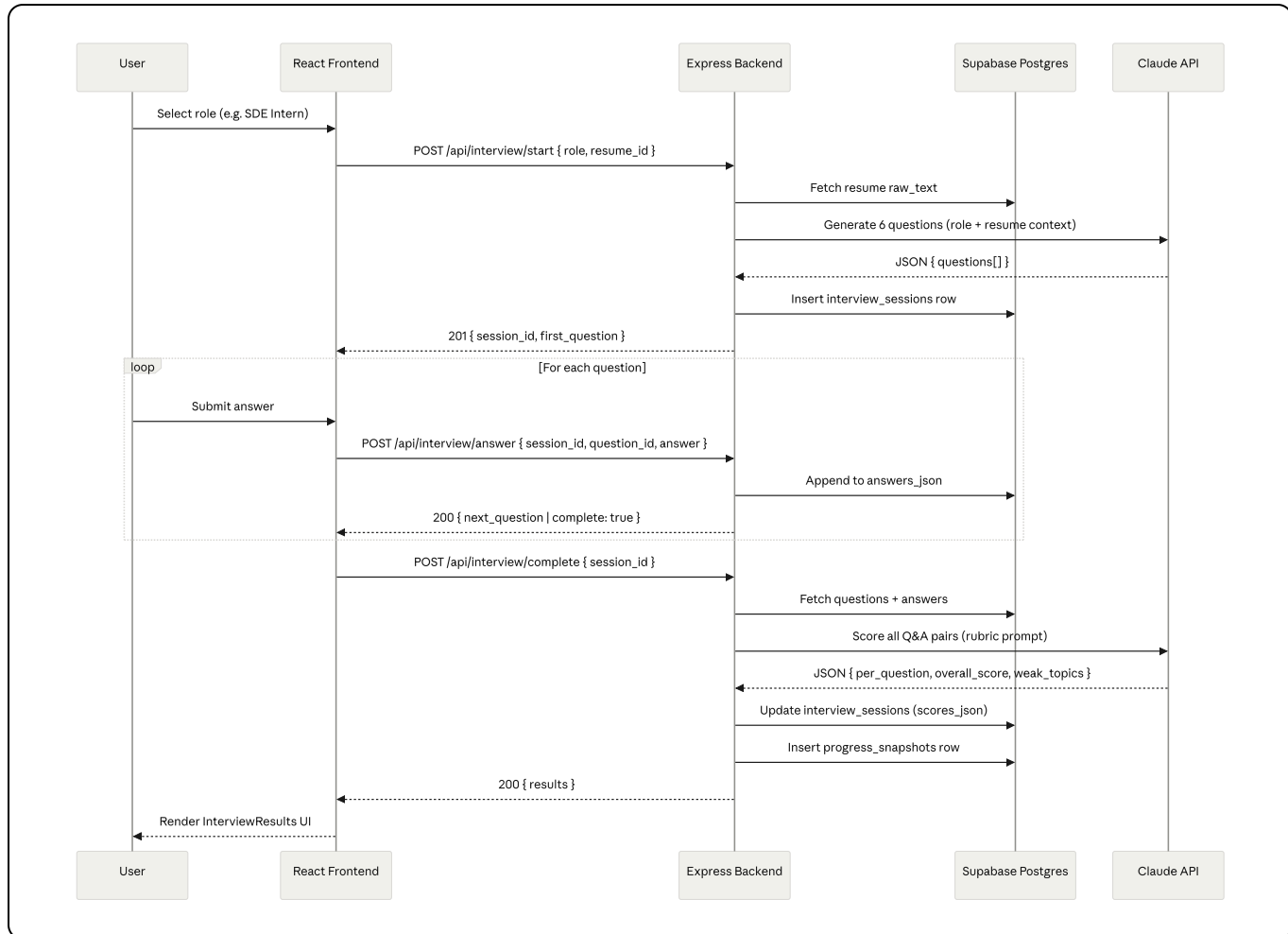
2. Component Diagram (Mermaid)



3. Data Flow — Resume Analysis (Sequence)



4. Data Flow — Mock Interview (Sequence)



5. Request Lifecycle (Every Authenticated API Call)

1. User action in React triggers an API call via a shared `apiClient.js` (axios/fetch wrapper).
2. `apiClient.js` attaches `Authorization: Bearer <supabase_jwt>` from the current session automatically.
3. Request hits Express route → passes through `authMiddleware.js`.
4. `authMiddleware.js` verifies the JWT using Supabase's public key/`supabase.auth.getUser(token)`; attaches `req.user` on success, else returns `401`.
5. Route handler executes business logic, calling `services/` layer (AI service, parsing service, DB queries via Supabase server client with service-role key).
6. Response returned as consistent JSON envelope: `{ success: boolean, data?: any, error?: string }`.
7. Frontend's `apiClient.js` centrally handles error responses → triggers toast notifications, no per-component try/catch duplication.

6. AI Interaction Design

- All AI calls happen **server-side only** — the Anthropic API key is never exposed to the client.
- Two AI service functions encapsulate all LLM interaction:

- `analyzeResume(resumeText, targetRole)` → resume feedback JSON
- `generateQuestions(role, resumeText)` / `scoreInterview(questions, answers)` → interview JSON
- All prompts enforce **strict JSON-only output** with an explicit schema in the system prompt.
- A shared `safeParseAIJson(rawText)` utility strips markdown code fences and validates against an expected shape before returning to the route handler; on failure, it retries the AI call once with a "Your last response was not valid JSON, return ONLY JSON" corrective follow-up.
- Temperature is tuned per use case: 0.3 for scoring (consistency), 0.6–0.7 for question generation (variety).

7. External Services Summary

Service	Role	Free Tier Limit (approx.)
Supabase	Auth, Postgres DB, Storage	500MB DB, 1GB storage, 50k monthly active users
Anthropic Claude API	Resume analysis, question generation, scoring	Pay-per-token; student should apply for API credits/promo where available
Vercel	Frontend hosting	Unlimited personal projects, generous bandwidth
Render	Backend hosting	Free web service (spins down on inactivity — acceptable for a portfolio demo)

8. Key Architectural Decisions & Trade-offs

Decision	Alternative Considered	Why This Choice Won
Express backend proxy for AI calls	Calling Claude directly from frontend	Frontend calls would expose API key; server-side keeps it secret and allows validation/retry logic
Supabase over self-hosted Postgres	Raw Postgres + custom auth	Supabase bundles Auth + Storage + DB, drastically reduces Day 2 setup time
JSON-schema-enforced prompts	Free-form AI text parsed with regex	Regex parsing of free text is fragile; strict JSON prompting is far more reliable for UI rendering
Render free tier despite cold-starts	Railway / Fly.io	Render's free tier is well-documented and sufficient for a demo; cold-start delay is an acceptable, explainable trade-off for v1